



Interview Preparation Guide for Rohit Garwad

Face-to-Face Technical Round + HR Round

Congratulations on clearing Rounds 1 & 2! 🎉

PART 1: TECHNICAL ROUND (Resume-Based)

Your Elevator Pitch (30 seconds - 1 minute)

"Hi, I'm Rohit Garwad, a Full-Stack and AI Engineer with hands-on experience in building web applications and AI-powered systems. I completed my MSc in Computer Science where I was the first-year topper. During my internship at Edunet Foundation, I built full-stack applications using React and Spring Boot. Recently, I've been focusing on Agentic AI and GenAI, building projects like an AI-powered App Builder using LangChain and LangGraph. I have 57+ repositories on GitHub and hold a 5-Star Gold Badge on HackerRank for problem-solving. I'm excited about this opportunity because it aligns with my passion for building scalable applications."

Questions About Your Experience

Q1: Tell me about your internship at Edunet Foundation.

Answer: "During my 6-month internship at Edunet Foundation (Nov 2023 - Apr 2024), I worked on building a full-stack web application as part of an internal capstone project."

My responsibilities included:

- Building the frontend using React with Redux for state management
- Developing REST APIs using Spring Boot
- Integrating both MySQL and MongoDB databases
- Implementing responsive UI using Tailwind CSS
- Using Git for version control and following Agile practices

Key achievement: I improved backend reliability by structuring the API layer properly and implementing error handling patterns.

Learning: This experience taught me how to work in a team environment, follow coding standards, and deliver production-ready code."

Q2: You have limited work experience. How will you handle real-world challenges?

Answer: "While I have one formal internship, I've compensated by building **57+ projects** on GitHub that simulate real-world challenges:

1. **My Project Management System** handles real scenarios like role-based access, real-time updates with WebSockets, and team collaboration
2. **My AI projects** deal with complex challenges like multi-agent orchestration, RAG pipelines, and handling LLM responses
3. **HackerRank 5-Star Badge** shows I can solve algorithmic problems efficiently

I'm a fast learner - I taught myself LangChain, LangGraph, and Agentic AI concepts within months and built production-ready projects. I believe my strong foundation and ability to learn quickly will help me adapt to any real-

world challenge."

Questions About Your Projects

Q3: Explain your App-Builder project. How does it work?

Answer: "App-Builder is my most complex AI project - it's an AI-powered application generator that creates multi-file project structures automatically."

How it works:

1. **User Input:** User describes what app they want in natural language
2. **Planner Agent:** Analyzes requirements, breaks them into tasks with dependencies
3. **Architect Agent:** Designs the folder structure and tech stack
4. **Coder Agent:** Generates code file-by-file following the plan
5. **Output:** Complete, runnable project with all files

Technical Stack:

- Python for backend
- LangChain for LLM orchestration
- LangGraph for managing agent state and workflows
- FastAPI for the API layer
- React for the Web UI
- Supports multiple LLMs (Groq, OpenAI, Gemini)

Key Challenges I Solved:

- **Dependency management:** Tasks execute in correct order
- **State management:** Using LangGraph to maintain conversation state
- **Modular design:** Each agent is independent and testable
- **Error handling:** Graceful handling of LLM failures

This project demonstrates my understanding of agentic AI patterns and system design."

Q4: Explain your Project Management System.

Answer: "This was my MSc final year project - a full-stack application for team collaboration."

Features:

- **Role-based access control:** Admin, Project Manager, Developer roles with different permissions
- **Real-time updates:** Using WebSockets so task changes reflect immediately for all users
- **Task management:** Create, assign, track, and complete tasks
- **Team collaboration:** Project dashboards, notifications, task comments
- **JWT Authentication:** Secure login system

Tech Stack:

- **Frontend:** React.js, Redux (state), Tailwind CSS, Shadcn UI
- **Backend:** Spring Boot, Node.js
- **Database:** MySQL
- **Real-time:** WebSockets

Architecture Decisions:

- Chose React for component reusability
- Used Redux for predictable state management across components
- Implemented JWT for stateless authentication
- Used WebSockets instead of polling for real-time features (more efficient)

Learning: This project taught me end-to-end development, from database design to deployment."

Q5: Explain your Cold Email Generator project.

Answer: "This is a RAG-based GenAI application that helps services companies send personalized cold emails.

How it works:

1. User provides a company's career page URL
2. System scrapes and extracts job listings
3. For each job, it matches relevant portfolio links from a vector database
4. LLM generates a personalized cold email with those links

Technical Implementation:

- **Web Scraping:** Extract job descriptions from career pages
- **Vector Database:** ChromaDB stores portfolio links with embeddings
- **RAG Pipeline:** Query vector DB to find relevant portfolio items
- **LLM (Groq):** Generates personalized email content
- **Streamlit:** Simple UI for user interaction

Why RAG? Instead of hardcoding portfolio links, using vector search ensures the most relevant links are included based on semantic similarity to the job description."

Q6: What is RAG? Explain with an example from your project.

Answer: "RAG stands for **Retrieval-Augmented Generation**. It combines retrieval (searching a knowledge base) with generation (LLM creating responses).

Why RAG?

- LLMs have knowledge cutoffs and can hallucinate
- RAG grounds responses in actual data

How I used it in Cold Email Generator:

1. **Indexing:** Portfolio links are converted to embeddings and stored in ChromaDB
2. **Retrieval:** When generating an email, the job description is embedded and used to search for similar portfolio items
3. **Augmentation:** Retrieved portfolio links are added to the LLM prompt
4. **Generation:** LLM generates email with actual, relevant links

Example:

- Job: 'Looking for Python developer with ML experience'
- Retrieved: My ML projects, Python projects from portfolio
- Generated email includes these relevant links

This ensures emails are personalized and accurate, not generic."

Q7: What is LangChain and LangGraph? Why did you use them?

Answer: "**LangChain** is a framework for building LLM-powered applications. It provides:

- Chains for sequencing LLM calls
- Prompt templates
- Memory for conversations
- Tools and agents
- Vector store integrations

LangGraph extends LangChain for building stateful, multi-actor applications:

- Represents workflows as graphs
- Manages state across multiple agents
- Handles conditional logic and loops
- Better for complex, multi-step workflows

Why I used them in App-Builder:

- **LangChain:** For LLM interactions, prompt management, tool calling
- **LangGraph:** For orchestrating multiple agents (Planner → Architect → Coder) with proper state management

LangGraph was essential because my agents need to share state (the project plan, file structure) and execute in a specific order with conditional paths."

Q8: Explain WebSockets. Why did you use them in your Project Management System?

Answer: "**WebSocket** is a protocol that provides full-duplex, persistent communication between client and server over a single connection.

HTTP vs WebSocket:

HTTP	WebSocket
Request-Response	Bi-directional
Connection closes after response	Connection stays open
Client initiates	Both can send anytime

Why I used WebSockets: When one user updates a task, all team members should see the change immediately. Without WebSockets, I'd need:

- Polling (inefficient, delayed updates)
- Long polling (still not ideal)

With WebSockets:

1. Client connects once
2. Server pushes updates instantly when any task changes
3. All connected clients receive updates in real-time

Implementation:

```
// Server broadcasts when task updates
io.emit('taskUpdated', { taskId, newStatus });
```

```
// All clients receive
socket.on('taskUpdated', (data) => {
    updateTaskInUI(data);
});
```

This gave users a smooth, real-time collaborative experience."

Technical Skill Questions

Q9: Tell me about your Java/Spring Boot experience.

Answer: "I've used Java and Spring Boot extensively:

During Internship:

- Built REST APIs with Spring Boot
- Implemented JWT authentication
- Connected to MySQL using Spring Data JPA
- Used Spring Security for authorization

In Project Management System:

- Created microservices-style backend
- Implemented CRUD operations for projects, tasks, users
- Used Spring Boot's dependency injection
- Wrote unit tests

Key Spring Boot Concepts I'm comfortable with:

- Dependency Injection / IoC
- REST Controllers and RequestMapping
- JPA/Hibernate for database operations
- Spring Security for auth
- Exception handling with @ControllerAdvice
- Configuration with application.properties

I'm also familiar with Java core concepts like OOP, Collections, Streams, and Multi-threading."

Q10: Explain REST API best practices you follow.

Answer: "I follow these REST API best practices:

1. Proper HTTP Methods:

- GET for reading
- POST for creating
- PUT/PATCH for updating
- DELETE for removing

2. Meaningful URLs:

- Nouns, not verbs: `/api/users` not `/api/getUsers`
- Hierarchical: `/api/projects/:projectId/tasks`

3. Status Codes:

- 200 for success
- 201 for created
- 400 for bad request
- 401 for unauthorized
- 404 for not found
- 500 for server errors

4. Consistent Response Format:

```
{
  "success": true,
  "data": { ... },
  "message": "User created successfully"
}
```

5. **Versioning:** /api/v1/users

6. **Authentication:** JWT tokens in Authorization header

7. **Error Handling:** Meaningful error messages with consistent structure

8. **Pagination:** For list endpoints with limit/offset or page/size"

Q11: Explain how you've used React in your projects.

Answer: "I've built multiple React applications including my portfolio and Project Management System.

Key React concepts I use:

1. Functional Components with Hooks:

- useState for local state
- useEffect for side effects (API calls)
- useContext for global state (auth, theme)
- useRef for DOM access
- useMemo/useCallback for optimization

2. State Management:

- React Context for simpler apps
- Redux for complex state (used in PMS)

3. Routing:

- React Router for navigation
- Protected routes for authentication

4. Component Patterns:

- Container/Presentational pattern
- Custom hooks for reusable logic
- Composition over inheritance

5. Styling:

- Tailwind CSS for utility-first styling
- Shadcn UI for component library

Example from PMS:

```
// Custom hook for fetching tasks
function useTasks(projectId) {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchTasks(projectId).then(setTasks).finally(() => setLoading(false));
  }, [projectId]);

  return { tasks, loading };
}
```

```

### Q12: What databases have you worked with? SQL vs NoSQL?

**Answer:**

"I've worked with both:

**SQL (MySQL, PostgreSQL):**

- Project Management System
- Food Ordering Chatbot
- Various internship projects

**NoSQL (MongoDB):**

- Internship projects
- Used for flexible schema requirements

**When to use what:**

| SQL                         | NoSQL                        |
|-----------------------------|------------------------------|
| Structured, relational data | Flexible, changing schema    |
| Complex queries with joins  | Simple queries, denormalized |
| ACID compliance critical    | Horizontal scaling needed    |
| Financial transactions      | Real-time analytics, logs    |

**Example decision in PMS:**

I chose MySQL because:

- Data is highly relational (Users → Projects → Tasks)
- Need complex queries (tasks by project by user by date)
- Transactions needed for task assignments
- Data integrity is important

For my AI projects, I use vector databases like ChromaDB and FAISS for embedding storage and similarity search."

---

### Q13: What is JWT? How did you implement authentication?

**\*\*Answer:\*\***

**JWT (JSON Web Token)** is a compact, self-contained token for securely transmitting information.

**\*\*Structure:\*\***

header.payload.signature

- **\*\*Header:\*\*** Algorithm, token type
- **\*\*Payload:\*\*** User data (id, email, role, expiry)
- **\*\*Signature:\*\*** Verification hash

**\*\*How I implemented it:\*\***

**\*\*Registration/Login:\*\***

1. User submits credentials
2. Server validates and creates JWT with user id
3. JWT sent to client
4. Client stores in localStorage/cookie

**\*\*Protected Routes:\*\***

1. Client sends JWT in Authorization header: `Bearer <token>`
2. Server middleware extracts and verifies token
3. If valid, adds user to request object
4. If invalid, returns 401

**\*\*Code Example:\*\***

```
``javascript
// Create token
const token = jwt.sign({ id: user._id }, SECRET, { expiresIn: '7d' });

// Verify token (middleware)
const decoded = jwt.verify(token, SECRET);
req.user = await User.findById(decoded.id);
```

#### Security Considerations:

- Short expiry time
- HTTPS only
- Don't store sensitive data in payload
- Implement refresh tokens for better UX"

---

## Problem-Solving Questions

**Q14: Describe a challenging problem you solved.**

**Answer (STAR Method):**



**Situation:** "In my App-Builder project, I faced a complex challenge: ensuring AI agents execute tasks in the correct order based on dependencies. For example, you can't code a component until its parent module is planned."

**Task:** "I needed to implement a dependency-aware task execution system that could handle complex project structures with multiple interdependent files."

**Action:** "I implemented a solution using:

1. **Graph-based dependency tracking:** Represented tasks as nodes with edges for dependencies
2. **Topological sorting:** Ensured correct execution order
3. **LangGraph state management:** Tracked completed tasks and unlocked dependent ones
4. **Parallel execution:** Tasks without dependencies could run simultaneously

```
def get_next_tasks(state):
 completed = state['completed_tasks']
 all_tasks = state['task_plan']

 ready = [
 task for task in all_tasks
 if task not in completed
 and all(dep in completed for dep in task.dependencies)
]
 return ready
"""

Result:
"The system now correctly handles complex project structures, generating 20+ file projects
with proper ordering. This reduced generation errors by 90% and made the output reliable."

Q15: How do you approach learning new technologies?

Answer:
"I follow a structured approach:

1. **Understand the 'Why':** Why does this technology exist? What problem does it solve?

2. **Official Documentation:** Start with official docs and tutorials

3. **Build Something Small:** Create a mini-project to apply concepts

4. **Build Something Real:** Apply to a meaningful project

5. **Teach/Document:** Writing about it solidifies understanding

Example - Learning LangGraph:
1. Understood it's for complex, stateful agent workflows
2. Read official docs and watched tutorials
3. Built a simple chatbot with state
4. Applied it to App-Builder for multi-agent orchestration
5. Now I can explain it to others
```

This approach helped me learn LangChain, LangGraph, Spring Boot, React, and more within short timeframes."

---

## ## Questions to Ask Them

1. "What does a typical day look like for this role?"
2. "What tech stack does the team primarily work with?"
3. "How is the team structured? Who would I be working with directly?"
4. "What are the biggest challenges the team is currently facing?"
5. "What opportunities are there for learning and growth?"
6. "What are the next steps in the interview process?"

---

## # PART 2: HR ROUND

### ## Common HR Questions

### Q1: Tell me about yourself.

**\*\*Answer:\*\***

"I'm Rohit Garwad, a Full-Stack and AI Engineer from Hyderabad. I completed my MSc in Computer Science where I was the first-year topper with a GPA of 8.31.

I have hands-on experience building web applications during my internship at Edunet Foundation, where I worked with React, Spring Boot, and databases.

What excites me most is the intersection of AI and software development. I've built several AI-powered projects including an App-Builder that uses LLMs to generate complete project structures.

I'm known for being a fast learner - I have 57+ repositories on GitHub and hold a 5-Star Gold Badge on HackerRank.

I'm looking for a role where I can apply my full-stack and AI skills to build impactful products while continuing to grow as an engineer."

---

### Q2: Why do you want to work here?

**\*\*Answer:\*\***

"I want to work here for three reasons:

1. **\*\*Technical Growth:\*\*** Your company works on [mention their tech/products], which aligns with my skills in full-stack development and AI. I'm excited to learn from experienced engineers and work on challenging problems.
2. **\*\*Impact:\*\*** I want to build products that users actually use. Your company's [mention

their product/mission] resonates with me because [specific reason].

3. **Culture:** From my research, I can see that you value [innovation/learning/teamwork - whatever you've learned about them]. This matches how I like to work - I'm collaborative, always learning, and focused on delivering results.

This role perfectly combines my experience with my career goals."

---

### Q3: What are your strengths?

**Answer:**

"My three key strengths are:

1. **Fast Learner:** I taught myself LangChain, LangGraph, and Agentic AI concepts in months and built production-ready projects. When I need to learn something, I dive deep and apply it quickly.

2. **Problem-Solving:** My 5-Star HackerRank badge demonstrates my analytical skills. I approach problems systematically - understand, break down, solve, optimize.

3. **Self-Motivated:** With 57+ GitHub repositories, I'm constantly building and learning. I don't wait to be told what to do - I identify what needs to be done and do it.

These strengths helped me successfully complete my internship and build complex projects independently."

---

### Q4: What are your weaknesses?

**Answer:**

"One area I'm actively improving is **communication in** large groups. While I'm comfortable in one-on-one discussions or small teams, I sometimes hold back in larger meetings.

**What I'm doing about it:**

- I prepare talking points before meetings
- I've started volunteering to present in discussions
- I'm taking opportunities to explain my projects to others

I've already seen improvement - I was able to present my final year project to my entire class confidently.

Another area is **saying no to interesting projects**. I get excited about new technologies and sometimes take on too much. I'm learning to prioritize and focus on completing one thing well before starting another."

---

### Q5: Where do you see yourself in 5 years?

**\*\*Answer:\*\***

"In 5 years, I see myself as a **\*\*Senior Full-Stack/AI Engineer\*\*** who:

1. **\*\*Technically:\*\*** Has deep expertise in building scalable applications and AI systems. I want to be someone the team comes to for architectural decisions.
2. **\*\*Leadership:\*\*** Has started mentoring junior developers and contributing to technical direction.
3. **\*\*Impact:\*\*** Has shipped multiple products that users rely on daily.

I plan to achieve this by:

- Mastering the fundamentals in my first 1-2 years
- Taking ownership of increasingly complex projects
- Learning from senior engineers and best practices
- Contributing to team knowledge through documentation and mentoring

This company seems like a great place to achieve these goals."

---

**### Q6: Why should we hire you?**

**\*\*Answer:\*\***

"You should hire me for three reasons:

1. **\*\*Relevant Skills:\*\*** I have hands-on experience with your tech stack - React, Java/Spring Boot, Node.js, and databases. Plus, I bring AI/GenAI skills that are increasingly valuable.
2. **\*\*Proven Track Record:\*\*** I cleared your technical rounds. My internship experience, 57+ projects, and 5-Star HackerRank badge show I can deliver quality code.
3. **\*\*Growth Mindset:\*\*** I'm a fast learner who's constantly improving. I'll ramp up quickly and start contributing. And I genuinely want to be here - this isn't just a job for me.

I'm ready to hit the ground running and make meaningful contributions to your team."

---

**### Q7: Describe a time you worked in a team.**

**\*\*Answer (STAR Method):\*\***

**\*\*Situation:\*\***

"During my internship at Edunet Foundation, I was part of a 4-person team building a web application."

**\*\*Task:\*\***

"My responsibility was the backend API development, but I needed to coordinate closely with the frontend developer to ensure smooth integration."

**\*\*Action:\*\***

"I took several steps:

1. **\*\*Clear API documentation:\*\*** Created detailed API specs before coding so frontend knew what to expect
2. **\*\*Regular sync-ups:\*\*** Had daily 15-minute check-ins with the team
3. **\*\*Helping others:\*\*** When a teammate struggled with React state management, I paired with them to solve it
4. **\*\*Git workflow:\*\*** Followed branching strategy and code review process"

**\*\*Result:\*\***

"We delivered the project on time with minimal integration issues. My manager specifically appreciated how the API documentation reduced back-and-forth communication. This taught me that good communication prevents problems."

---

### Q8: Describe a time you faced conflict.

**\*\*Answer (STAR Method):\*\***

**\*\*Situation:\*\***

"During a college group project, a teammate and I disagreed on the database choice - I wanted MySQL, he wanted MongoDB."

**\*\*Task:\*\***

"We needed to resolve this quickly without affecting team morale or project timeline."

**\*\*Action:\*\***

"Instead of arguing, I suggested we:

1. List pros and cons of each for our specific use case
2. Consider factors: data structure, query complexity, team familiarity
3. Make decision based on project needs, not personal preference

We realized our data was highly relational with complex queries, making MySQL the better choice. I also acknowledged that MongoDB could be right for a different project."

**\*\*Result:\*\***

"We made a data-driven decision together. My teammate felt heard, and we moved forward unified. I learned that focusing on objective criteria removes personal conflict from technical decisions."

---

### Q9: How do you handle pressure/deadlines?

**\*\*Answer:\*\***

"I handle pressure through **\*\*preparation and prioritization\*\***:

1. **\*\*Break it down:\*\*** Large tasks feel overwhelming. I break them into smaller, manageable pieces with mini-deadlines.
2. **\*\*Prioritize ruthlessly:\*\*** Focus on what's critical first. Not everything needs to be perfect - some things just need to work.

3. **Communicate early:** If I see a risk of missing a deadline, I communicate early so we can adjust scope or get help.

4. **Stay calm:** Stress leads to mistakes. I take short breaks to maintain clarity.

**Example:** For my final year project deadline, I had 2 weeks to complete a complex feature. I:

- Listed all tasks and estimated time
- Prioritized core functionality over nice-to-haves
- Worked focused 4-hour blocks with breaks
- Delivered the core feature on time, added extras the next week

I actually perform well under pressure because it forces focus."

---

### Q10: What are your salary expectations?

**Answer:**

"I'm focused on finding the right opportunity where I can learn and grow. Based on my research of the market and considering my skills and experience, I'm looking for something in the range of [X to Y].

However, I'm flexible and open to discussing this based on the complete compensation package and growth opportunities. What's the budget you have in mind for this role?

[Research typical fresher salaries for MERN/Full-stack roles in Hyderabad - usually 3-6 LPA for freshers]"

**Tips:**

- Research market rates before the interview
- Give a range, not a fixed number
- Show flexibility
- Ask about their budget
- Consider total package (base + bonus + benefits)

---

### Q11: Do you have any questions for us?

**Always ask questions! It shows interest.**

1. "What would success look like in this role in the first 6 months?"
2. "What's the team culture like? How do you handle collaboration?"
3. "What learning and development opportunities are available?"
4. "What are the next steps in the process?"
5. "Is there anything about my background that gives you hesitation that I can address?"

---

## Final Tips for Both Rounds

### ### Before the Interview:

- ✅ Research the company (products, tech stack, recent news)
- ✅ Review your resume - be ready to explain everything
- ✅ Prepare your elevator pitch
- ✅ Dress professionally
- ✅ Arrive 10-15 minutes early
- ✅ Bring copies of your resume

### ### During the Interview:

- ✅ Make eye contact, smile, be confident
- ✅ Listen carefully before answering
- ✅ Take a moment to think if needed
- ✅ Use STAR method for behavioral questions
- ✅ Be honest - don't bluff
- ✅ Show enthusiasm

### ### If You Don't Know Something:

Say: "I haven't worked with that directly, but based on my understanding of [related concept], I would approach it by... I'm definitely interested in learning more about it."

### ### Body Language:

- Sit up straight
- Don't cross arms
- Nod to show you're listening
- Mirror their energy level

---

## ## Quick Reference Card

### \*\*Your Key Selling Points:\*\*

- 🎓 MSc CS, First-Year Topper
- 💼 Internship at Edunet Foundation
- 🖥️ AI/GenAI + Full-Stack skills
- 📁 57+ GitHub repositories
- ⭐ 5-Star HackerRank

### \*\*Your Top Projects to Mention:\*\*

1. App-Builder (Agentic AI)
2. Project Management System (Full-Stack)
3. Cold Email Generator (RAG/GenAI)

### \*\*Your Tech Stack:\*\*

- Frontend: React, Redux, Tailwind
- Backend: Spring Boot, Node.js, Express
- Databases: MySQL, MongoDB, Vector DBs
- AI: LangChain, LangGraph, LLMs

---

**\*\*You've got this! You already cleared the hard rounds. Be confident! 🚀\*\***

Good luck! 🍀